Automated Irregular Shaped Object Stacking via Pose Optimization and Monte Carlo Tree Search

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—This paper presents a novel approach for automated stacking of irregularly shaped objects. The proposed method combines Monte Carlo Tree Search (MCTS) with optimization techniques to achieve stable and efficient stacking configurations.**

I. Introduction

Automating the process of stacking objects has long been studied in the context of packing and warehousing systems. Our contributions are as follows:

•

The rest of the paper is organized as follows. Sec. II Sec. III Sec. IV Sec. V

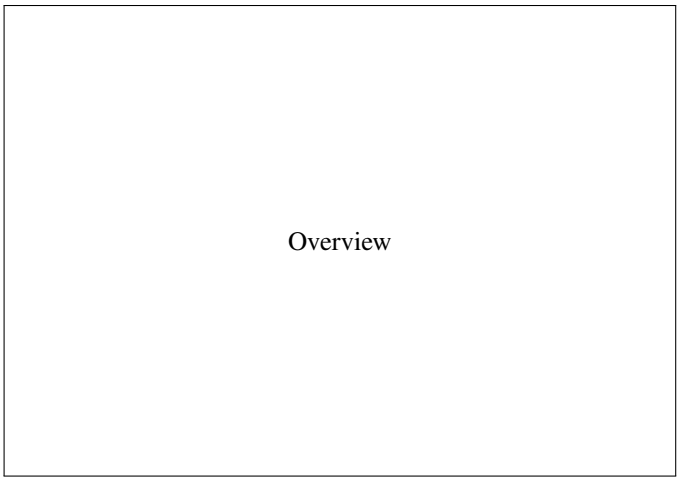


Fig. 1: Overview.

II. Related Works

III. Overview and Preliminaries

A. Stacking as an Optimization Problem

Suppose that we are given a target shape $T \subset \mathbb{R}^3$ and n irregular shaped objects $O_1(q_1), O_2(q_2), \cdots, O_n(q_n) \subset \mathbb{R}^3$ where each $q_i \in \text{SE}(3)$ represents the pose of each object O_i in the world frame. The goal is to find optimal placement poses $q_1, \cdots, q_n$ for a set of irregularly shaped objects such that the final configuration approximates the target shape T while satisfying the stability and non-collision constraints. This problem can be formulated as an optimization problem:

$$\min_{q_1,\cdots,q_n \in \text{SE}(3)} \mathbb{V}\left(T \setminus \bigcup_{i=1}^{n} O_i(q_i)\right) \quad \text{s.t.} \quad c_{\text{phys}}(O_1(q_1), \cdots, O_n(q_n)) = 0 \tag{1}$$

where $\mathbb{V}(\cdot)$ is the volume of a 3D shape and $c_{\text{phys}}(\cdot)$ is a function that evaluates the physical feasibility of the configuration, including stability and non-collision constraints.

B. Shape Representation

Throughout the paper, we use the union of differentiable support function (DSF) to represent the 3D shapes of the target shape T and the objects O_i, $i \in \mathcal{I}_k$, in the inventory. Given a compact convex body $C \subset \mathbb{R}^3$, the support function $h_C : \mathbb{R}^3 \setminus \{0\} \to \mathbb{R}$ is defined as

$$h_C(x) = \max_{s \in C} x \cdot s$$

where $\cdot$ denotes the standard inner product in $\mathbb{R}^3$. The work of Lee et al. [9] presented a differentiable version of the support function h_C that represents a smooth shape C.

The function $h_C(x)$ can be easily extended through rigid-body transformation as follows:

$$h_{\bar{C}(q)}(x) = h_C(R^\top x) + t \cdot x$$

for $q := (R, t) \in \text{SE}(3)$ where $\bar{C}(q) = \{Rp + t \mid p \in C\}$ denotes a transformed C under rigid-body transformation. Given two DSF represented convex bodies $C_1(q_1)$ and $C_2(q_2)$ with SE(3) poses q_1, q_2, the signed distance $\mathrm{d}(C_1(q_1), C_2(q_2))$ between the two convex bodies can be easily computed using Riemannian optimization techniques associated with $h_{C_1(q_1)}$ and $h_{C_2(q_2)}$ as presented in An, et al. [2]. Moreover, due to the smoothness of DSFs, the signed distance $\mathrm{d}(C_1(q_1), C_2(q_2))$ and the other contact information (e.g., normal vectors and contact points) are differentiable with respect to q_1 and q_2, which allows us to use gradient-based optimization techniques.

C. Markov Decision Process Formulation

Nevertheless, solving the optimization problem in Eq. (1) is extremely challenging due to the high-dimensional search space as the number of the objects n increases and the presence of complex physical constraints that involve non-convex and non-differentiable functions.

We model the construction as a finite-horizon sequential decision process in which one object is placed at each step. Let $\mathcal{X}_k = \{(i, q_i) \mid i \in \mathcal{P}_k\}$ be the current scene and $\mathcal{I}_k = \{1, \ldots, n\} \setminus \mathcal{P}_k$ the remaining inventory. The state, action, and deterministic transition function are

$$s_k = (\mathcal{X}_k, \mathcal{I}_k, T), \quad a_k = (i_{k+1}, q_{k+1}), \quad s_{k+1} = F(s_k, a_k),$$

where $i_{k+1} \in \mathcal{I}_k$. The transition inserts the proposed pose into the scene, simulates the resulting structure, and returns the settled scene with $\mathcal{I}_{k+1} = \mathcal{I}_k \setminus \{i_{k+1}\}$. The reward $r(s_k, a_k)$ measures construction progress and physical validity; its implementation is described in Sec. V. The sequence planner therefore operates on the MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, F, r, \gamma)$.

D. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) incrementally constructs a search tree $\mathcal{T} = (\mathcal{N}, \mathcal{E})$ over the MDP. Each node $n \in \mathcal{N}$ represents a state $s(n) \in \mathcal{S}$, and each edge $(n, a) \in \mathcal{E}$ represents an action $a \in \mathcal{A}$ and its transition to a child node. For a node or edge, let $N(n)$ denote the number of completed evaluations and let $Q(n)$ denote the empirical mean of the discounted returns backed up to that node. An action prior $P(n, a)$, distinct from the transition function F, can provide additional guidance when the action space is large or continuous.

Each MCTS iteration consists of four conceptual operations. During *selection*, the tree policy chooses a promising child by balancing its empirical value and an exploration bonus. During *expansion*, a new action is generated and attached to the selected node. During *evaluation*, the transition associated with the action is evaluated to obtain a reward and a successor state. Finally, during *backup*, the discounted return is propagated through the selected path to update $Q(n)$ and $N(n)$. A nonterminal leaf may additionally be assigned a leaf-value estimate $\hat{V}(s)$, typically obtained from rollout simulation or a problem-specific heuristic.

In the present problem, an action is a placement of one remaining object, and its evaluation includes pose refinement and physical simulation. The geometry-informed proposals, batched selection, and finite-horizon leaf evaluation used to

adapt MCTS to this expensive continuous-action setting are described in Sec. V.

E. Overall Structure of the Sequence Planner

The overall structure of the sequence planner is illustrated in Fig. 1. The sequence planner uses Monte Carlo Tree Search (MCTS) to explore the space of possible stacking sequences. For each expansion, it first uses the initial pose sampler π_{init} to generate a candidate pose $\hat{q}_{k+1}$, and then uses the pose optimization module π_{opt} to refine it into a locally optimal pose q_{k+1}. The structural simulation module π_{sim} subsequently implements the physical transition by evaluating the entire provisional scene and producing the settled next state. The resulting validity and simulation statistics are used to determine whether the action is admitted to the search tree and to evaluate its reward.

This gives the following execution flow:

$$s_k \xrightarrow{\pi_{\text{init}}} \hat{q}_{k+1} \xrightarrow{\pi_{\text{opt}}} q_{k+1} \xrightarrow{\pi_{\text{sim}}} s_{k+1},\ r(s_k, a_k).$$

We describe pose optimization in Sec. IV and sequence planning, including structural simulation and reward design, in Sec. V.

IV. Pose Optimization

Before presenting the other modules of our proposed sequence planner, we first introduce the pose optimization module π_{opt} that refines the candidate pose $\hat{q}_{k+1}$ generated by the initial pose sampler π_{init} to find a locally optimal placement pose q_{k+1} that is physically feasible and stable.

A. Single Object Pose Optimization Problem

Given s_k and the candidate pose $\hat{q}_{k+1}$ generated by π_{init}, the pose optimization module π_{opt} solves the following problem to obtain a locally optimal pose q_{k+1}:

$$\begin{aligned} \min_{q} \quad & \left[f_{\text{geo}}(q) + \min_{\lambda} f_{\text{phys}}(q, \lambda) \right] \\ \text{s.t.} \quad & g_{\text{cone},j}(q, \lambda_j) \le 0, \quad j = 1, \cdots, n_c \end{aligned} \tag{2}$$

where $q \in \text{SE}(3)$ is the pose of the object and $\lambda = (\lambda_1, \cdots, \lambda_{n_c}) \in \mathbb{R}^{3n_c}$ is the vector of contact forces at the n_c contact points between the object and the scene. This formulation follows prior contact-aware optimization methods [10, 5].

The geometric objective $f_{\text{geo}}(q)$ measures geometric quality, including target shape approximation and collision avoidance; its detailed formulation is provided in the appendix. The physical objective $f_{\text{phys}}(q, \lambda)$ is designed to evaluate whether the contact forces λ are physically feasible and contribute to a stable configuration. For indices $l, m \in \mathcal{P}_{k+1} := \mathcal{P}_k \cup \{i_{k+1}\}$ such that $l \neq m$, let $I_{l,m} \subset \{1, \cdots, n_c\}$ be the set of contact point indices associated with the objects O_l, O_m, and let $I_l := \bigcup_{m \in \mathcal{P}_{k+1} \setminus \{l\}} I_{l,m}$ be the set of contact point indices associated with the object O_l. Then, the physical objective $f_{\text{phys}}(q, \lambda)$ is defined as the following:

$$\begin{aligned} f_{\text{phys}}(q, \lambda) = w_{\text{eq}} \sum_{l \in \mathcal{P}_{k+1}} & \| J_{c,I_l}^{\top}(q) \lambda_{I_l} + \lambda_{\text{ext},l} \|^2 \\ & + w_{\text{cp}} \sum_{j \in I_{i_{k+1}}} \mathrm{d}_j(q)^2 \|\lambda_j\|^2 + \rho \sum_{j=1}^{n_c} \|\lambda_j\|^2 \end{aligned} \tag{3}$$

The three terms respectively penalize violations of static equilibrium, contact complementarity between the candidate and the current scene, and large contact forces through regularization. Here, J_{c,I_l} is the contact Jacobian for object O_l, $\lambda_{\text{ext},l}$ is its external wrench, λ_{I_l} is the vector of contact forces at the contact points in I_l, and $w_{\text{eq}}, w_{\text{cp}}, \rho > 0$ are weighting and regularization parameters.

Finally, each $g_{\text{cone},j}(\cdot)$ is the smoothed friction cone constraints for the j-th contact point defined by

$$g_{\text{cone},j}(q, \lambda_j) = \sqrt{\|J_{c,j}^t \lambda_j\|^2 + \epsilon} - \mu_j J_{c,j}^n \lambda_j \tag{4}$$

where $\epsilon > 0$ is a smoothing parameter, μ_j is the friction coefficient, and $J_{c,j}^t$ and $J_{c,j}^n$ are the tangential and normal components of the Jacobian of the contact constraint.

We note that, due to the smoothness of DSFs, the terms in the objective function dependent to q are formulated which allows us to use gradient-based optimization techniques.

B. Upper-Level Problem Optimization on SE(3) Manifold

Denoting $\lambda^*(q)$ as the optimal λ of the lower-level problem for a given pose q, we define the force dependent objective function as $f_{\text{phys}}(q) := f_{\text{phys}}(q, \lambda^*(q))$. Then, using the *Envelope theorem* [4], we can compute the gradient of $f_{\text{phys}}(q)$ with respect to q as follows:

$$\frac{df_{\text{phys}}(q)}{dq} = \frac{\partial f_{\text{phys}}(q, \lambda^*)}{\partial q} + \sum_{j=1}^{n_c} \nu_j^* \frac{\partial g_{\text{cone},j}(q, \lambda_j^*)}{\partial q}$$

where ν_j^* is the optimal dual variable associated with the j-th friction cone constraint $g_{\text{cone},j}(q, \lambda_j^*) \le 0$. Using the above gradient, we optimize the placement pose q on SE(3) manifold based on the Riemannian trust-region method [1] combined with BFGS Hessian approximation [7]. In addition, motivated by homotopy based optimization methods [], we designed the sequence of the weight parameters in $f_{\text{geo}}(q)$ and $f_{\text{phys}}(q)$ such as w_{eq}, w_{cp} and iteratively solve the bi-level optimization problem (2) to induce a better local minimum.

C. Lower-Level Problem Optimization with ADMM

For a fixed q, the lower-level problem is a convex optimization problem with respect to λ which can be efficiently solved. Moreover, exploiting the inherent structure of the lower-level problem, we can reformulate it to be separable with respect to the objects $\mathcal{P}_{k+1}$ in the scene and apply the alternating direction method of multipliers (ADMM) [3] to solve it in a scalable manner. For each contact force λ_j, we introduce two local copies $\lambda_{j,1}$ and $\lambda_{j,2}$ associated with the two objects in contact. Specifically, for $j \in I_{l,m}$ with $l < m$, $\lambda_{j,1}$ and

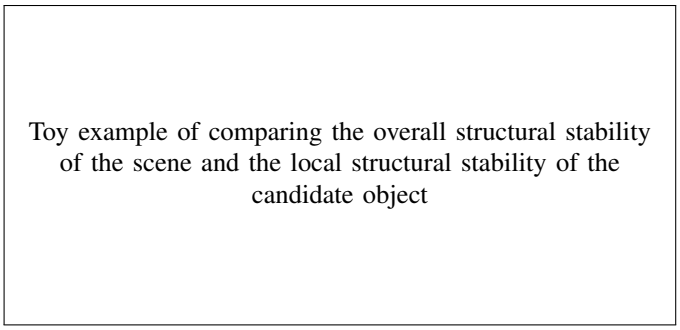


Fig. 2: Toy example of comparing the overall structural stability of the scene and the local structural stability of the candidate object.

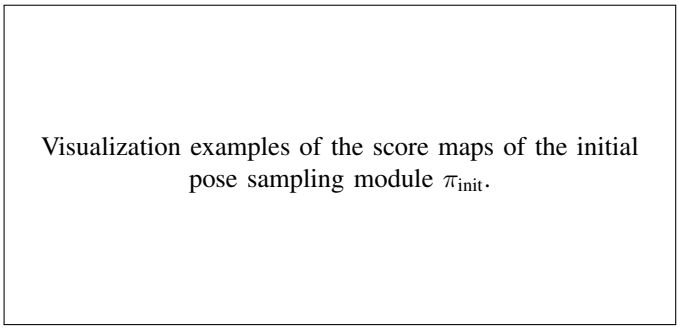


Fig. 3: Score maps of the initial pose sampling module π_{init}.

$\lambda_{j,2}$ are associated with O_l and O_m, respectively. We then impose the consensus constraints $\lambda_{j,1} = \lambda_{j,2}$. Collecting all local copies into $\tilde{\lambda}$, the first term in $f_{\text{phys}}(q, \lambda)$ can be written as an objectwise-separable sum. The lower-level problem can therefore be reformulated as

$$\begin{aligned} \min_{\tilde{\lambda}, z} \quad & \tilde{f}_{\text{phys}}(q, \tilde{\lambda}) + \sum_{\substack{j=1,\dots,n_c \\ s=1,2}} \delta_{K_j}(z_{j,s}) \\ \text{s.t.} \quad & \lambda_{j,1} = \lambda_{j,2} = z_{j,1} = z_{j,2}, \qquad j = 1, \dots, n_c. \end{aligned}$$

where $\tilde{f}_{\text{phys}}(q, \tilde{\lambda})$ is the extended physical force function and δ_{K_j} is the indicator function of the second-order cone $K_j := \{\lambda \in \mathbb{R}^3 \mid g_{\text{cone},j}(q, \lambda) \leq 0\}$. Here, $z_{j,s}$ is an auxiliary variable associated with the j-th contact point and $s \in \{1, 2\}$. At each ADMM iteration, the objectwise-separable subproblem decomposes into $|\mathcal{P}_{k+1}|$ independent subproblems, one for each object O_l. The remaining updates consist of $2n_c$ projections onto the second-order cones K_j which can be computed in closed form, and $2n_c$ auxiliary-variable updates, which reduce to averaging.

The pose optimization module is implemented using CUDA to leverage GPU parallelization for efficient computation of the gradients and the ADMM updates.

V. Sequence Planning

In this section, we introduce the remaining modules π_{init} and π_{sim} that are used to generate candidate poses and simulate the stability of the scene, respectively. Then, we present the Monte Carlo Tree Search (MCTS) algorithm that uses these modules to explore the space of possible stacking sequences and find an optimal sequence of object placements.

Algorithm 1 Initial pose sampling $\pi_{\text{init}}(s_k)$

Require: State $s_k = (\mathcal{X}_k, \mathcal{I}_k, T)$, rotation set $\mathcal{R}$
1: Sample $i_{k+1} \sim \text{Unif}(\mathcal{I}_k)$ and $R \sim \text{Unif}(\mathcal{R})$
2: Construct the feasible planar grid $\mathcal{G}_{i_{k+1},R}(s_k)$
3: Evaluate $\sigma(\mathbf{x} \mid i_{k+1}, R, s_k)$ for all $\mathbf{x} \in \mathcal{G}_{i_{k+1},R}(s_k)$
4: Sample $\mathbf{x}$ from $\pi_{xy}(\mathbf{x} \mid i_{k+1}, R)$
5: Set $\hat{q}_{k+1} = (R, (\mathbf{x}, z_{i_{k+1},R}(\mathbf{x}; s_k)))$
6: **return** $(i_{k+1}, \hat{q}_{k+1})$

A. Initial Pose Sampling

The initial pose sampler generates a candidate object, orientation, and planar translation from the current state $s_k = (\mathcal{X}_k, \mathcal{I}_k, T)$. Let $\mathcal{G}_{i,R}(s_k)$ denote the feasible planar sampling grid for object i and orientation R, and let $\hat{q}_{k+1} = (R, (\mathbf{x}, z_{i,R}(\mathbf{x}; s_k)))$. The sampler is defined by

$$\begin{aligned} &\pi_{\text{init}}(i, R, \mathbf{x} \mid s_k) = \pi_{xy}(\mathbf{x} \mid i, R, s_k)\pi_{\text{unif}}(i, R \mid s_k), \\ &\pi_{xy}(\mathbf{x} \mid i, R, s_k) \propto \exp(\sigma(\mathbf{x} \mid i, R, s_k)/\tau)\,. \end{aligned}$$

where $\pi_{\text{unif}}(i, R \mid s_k)$ is uniform over $i \in \mathcal{I}_k$ and the finite rotation set $\mathcal{R}$. The distribution π_{xy} is defined over $\mathbf{x} \in \mathcal{G}_{i,R}(s_k)$, and $\tau > 0$ controls the preference for higher-scoring planar locations. The initial height $z_{i,R}$ is computed from the scene geometry, while the full pose is subsequently refined by the pose optimization module π_{opt}. For brevity, the dependence of the score on s_k is omitted below. The proposal score $\sigma(\mathbf{x} \mid i_{k+1}, R)$ is formulated as a weighted sum of several geometric features that are computed based on the scene $\mathcal{X}_k$, the target shape T, and the object i_{k+1}:

$$\begin{aligned} \sigma(\mathbf{x} \mid i_{k+1}, R) = w_{\text{s}}\sigma_{\text{s}}(\mathbf{x} \mid i_{k+1}, R) + w_{\text{f}}\sigma_{\text{f}}(\mathbf{x} \mid i_{k+1}, R) \\ + w_{\text{b}}\sigma_{\text{b}}(\mathbf{x} \mid i_{k+1}, R) - w_{\text{h}}\sigma_{\text{h}}(\mathbf{x} \mid i_{k+1}, R), \end{aligned}$$

where σ_{s} is the support area score map, σ_{f} is the filled floor area score map, σ_{b} is the boundary proximity score map, and σ_{h} is the height score map, respectively with weight parameters w_{h}, w_{s}, $w_{\text{f}}, w_{\text{b}} > 0$.

Intuitively, the score σ_{s} encourages the object to be placed on a larger support area, the score σ_{f} encourages the object to fill the empty floor area, the score σ_{b} encourages to fill the boundary of the target shape T first, and the last score σ_{h} encourages the object to be placed at a lower height to improve stability. The visualization examples of the score maps are shown in Fig. 3 and the details of the score map computation will be described in the appendix.

B. Simulation Module

After π_{opt} refines the candidate pose $\hat{q}_{k+1}$, the simulation module π_{sim} evaluates the deterministic transition $s_{k+1} = F(s_k, a_k)$. It first simulates the new object with the existing scene fixed to refine the placement, and then simulates the entire scene to verify global stability.

Both stages use early termination. The simulation stops when all placed objects have sufficiently small kinetic energy or when accumulated motion energy indicates instability. For object O_i, we define

$$E_{\text{kin},i}(t) = \frac{1}{2m_i}\mathcal{V}_i^\top M_i \mathcal{V}_i, \quad E_{\text{acc},i}(t) = \sum_{s=0}^{t} \sqrt{E_{\text{kin},i}(s)}\Delta t$$

where $\mathcal{V}_i$ is the spatial velocity, M_i and m_i are the mass matrix and mass, respectively, and Δt is the simulation time step. The early-termination condition is

$$\max_{i\in\mathcal{P}_{k+1}} E_{\text{kin},i}(t) < \epsilon_{\text{kin}} \quad \text{or} \quad \max_{i\in\mathcal{P}_{k+1}} E_{\text{acc},i}(t) > \epsilon_{\text{acc}} \tag{5}$$

with the thresholds $\epsilon_{\text{kin}}, \epsilon_{\text{acc}} > 0$. We implement π_{sim} in CUDA using the GPU-accelerated subADMM algorithm [8].

C. Reward Design

The reward function is designed to encourage the agent to place objects in a way that maximizes the filled volume of the target shape T while ensuring the stability of the scene and formulated as follows:

$$\begin{aligned} r(s_k, a_k) = w_T \frac{\mathbb{V}\left(\overline{\mathcal{X}}_{k+1}\right) - \mathbb{V}\left(\overline{\mathcal{X}}_k\right)}{\mathbb{V}(T)} + w_O \frac{\mathbb{V}\left(O_{i_{k+1}} \cap T\right)}{\mathbb{V}\left(O_{i_{k+1}}\right)} \\ - w_H \left(\max_{\mathbf{x}} H_{\mathcal{X}_{k+1}}(\mathbf{x}) - \mathbb{E}_{\mathbf{x}}[H_{\mathcal{X}_{k+1}}(\mathbf{x})]\right) \\ - w_D \mathbb{E}_{\mathcal{V}\sim\mathcal{N}(0,\Sigma)}\left[\Delta q_{k+1}(\mathcal{V})\right] - w_S f_{\text{eq}} \end{aligned}$$

where $w_T, w_O, w_H, w_S, w_D > 0$ are the weight parameters for each reward term, $\overline{\mathcal{X}}_k := T \cap \bigcup_{i\in\mathcal{P}_k} O_i(q_i)$ is the intersection of the target shape T and the union of the placed objects in the scene $\mathcal{X}_k$, and $H_{\mathcal{X}_{k+1}}(\mathbf{x})$ is the uppper surface height of the scene $\mathcal{X}_{k+1}$ at planar location $\mathbf{x} = (x, y)$. The first term and the second term encourage the agent to place objects that fill the target shape T and maximize the filled volume of the target shape T, respectively. The third term penalizes the height difference between the maximum height and the average height of the scene $\mathcal{X}_{k+1}$ to encourage a more uniform distribution of objects in the scene. The fourth term penalizes the expected pose deviation $\Delta q_{k+1}(\mathcal{V})$ of the newly placed object $O_{i_{k+1}}$ under random perturbations $\mathcal{V}$ sampled from a Gaussian distribution $\mathcal{N}(0, \Sigma)$ to encourage the agent to place objects in a more stable manner. The last term penalizes the violation of the static equilibrium constraints f_{eq} from π_{opt} to encourage the agent to place objects in a physically feasible manner.

D. Monte Carlo Tree Search

We combine π_{init}, π_{opt}, and π_{sim} in a batched, receding-horizon MCTS planner over the MDP defined in Sec. III. To amortize expensive pose optimization and simulation, we evaluate candidate placements in parallel, following the batched planning approach of [6]. Algorithm 2 summarizes the finite-horizon search and execution procedure, starting from a tree rooted at the current state s_k.

Batched Selection. We select a batch of expansion paths using PUCT [11] with geometry- and equilibrium-informed action priors. For a child n reached from n_p by action a, selection maximizes

$$\text{PUCT}(n) = \frac{N(n)Q(n)}{N(n)+\hat{N}(n)} + cP(n_p, a)\frac{\sqrt{N(n_p)+\hat{N}(n_p)}}{1+N(n)+\hat{N}(n)}, \tag{6}$$

where Q is the mean return, N and $\hat{N}$ are completed and temporary virtual visit counts, P is the action prior, and $c > 0$ controls exploration. Progressive widening allows more children as completed visits increase, while virtual visits diversify selection within each batch. All paths are selected before physical evaluation begins.

Parallel Expansion and Evaluation. Each selected expansion applies π_{init}, π_{opt}, and π_{sim} to generate, refine, and physically evaluate a candidate placement. These evaluations run in parallel and provide settled successor states, validity outcomes, and rewards for tree expansion and backup. Root candidates are evaluated through the same pipeline.

Leaf Value Estimation and Backup. To avoid further optimization and simulation costs, we replace additional rollouts with a heuristic estimate of the remaining return. For a nonterminal leaf s_l at depth d_l from the current root, we use

$$\hat{V}(s_l) = \bar{r}\sum_{j=0}^{h_l-1}\gamma^j, \qquad h_l = \min\{d_M - d_l, |\mathcal{I}(s_l)|\},$$

where d_M is the maximum search depth, $\mathcal{I}(s_l)$ is the remaining inventory, and $\bar{r}$ is the mean reward estimated from evaluated placements. Terminal and failed leaves receive no bootstrap value. Discounted placement rewards and the leaf estimate are backed up along each selected path to update mean returns Q and visit counts N. Temporary virtual visits are then removed.

Action Selection and Tree Reuse. After the search budget is exhausted, we execute the physically validated root action with the highest mean return. In Algorithm 2, $\mathcal{A}_{\text{val}}(s_k)$ denotes these validated actions and n_a the child reached by a. The selected child becomes the new root, retaining descendants and

Algorithm 2 Batched receding-horizon MCTS for sequence planning

Require: State s_k, previous search tree $\mathcal{T}$, batch size B
Ensure: Next state s_{k+1} and updated tree $\mathcal{T}$

```
Initialize or update T at s_k and evaluate root candidates
while the search budget remains do
    B ← SELECTBATCH(T, B)
    E ← EVALUATEBATCH(B, π_init, π_opt, π_sim)
    T ← BACKUP(T, B, E)
end while
a* ← arg max_{a∈A_val(s_k)} Q(n_a)
Execute a* and obtain s_{k+1}
Promote its child to the root and retain compatible descendants
return s_{k+1}, T
```

search statistics compatible with the resulting state. Planning then repeats with a finite lookahead horizon.

ACKNOWLEDGMENTS

REFERENCES

[1] P. A. Absil, C. G. Baker, and K. A. Gallivan. Trust-region methods on riemannian manifolds. *Foundations of Computational Mathematics*, 7:303–330, 2007.

[2] S. An, S. Lee, J. Lee, S. Park, and D. Lee. Collision detection between smooth convex bodies via riemannian optimization framework. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12464–12471. IEEE, 2024.

[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

[4] M. Carter. *Foundations of mathematical economics*. MIT press, 2001.

[5] J. Chen, Y. Ke, and H. Wang. BODex: Scalable and efficient robotic dexterous grasp synthesis using bilevel optimization. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127930.

[6] Baichuan Huang, Abdeslam Boularias, and Jingjin Yu. Parallel monte carlo tree search with batched rigid-body simulations for speeding up long-horizon episodic robot planning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.

[7] W. Huang, P.-A. Absil, and K. A. Gallivan. A riemannian BFGS method without differentiated retraction for nonconvex optimization problems. *SIAM Journal on Optimization*, 28(1):470–495, 2018. doi: 10.1137/17M1127582.

[8] H. Ji, H. Kim, J. Lee, S. Lee, S. An, J. Heo, Y. Lee, Y. Lee, and D. Lee. Gpu-accelerated subsystem-based admm for large-scale interactive simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13868–13874. IEEE, 2025.

[9] J. Lee, M. Lee, and D. Lee. Uncertain pose estimation during contact tasks using differentiable contact features. *RSS 2023-Robotics: Science and Systems*, 2023.

[10] J. Lee, S. Park, M. Lee, and D. Lee. Efficient gradient-based inference for manipulation planning in contact factor graphs. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12464–12471. IEEE, 2025.

[11] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.